ModernTCN Source Code Walkthrough

Line-by-line mathematical annotations for
`ModernTCN.py`, `ModernTCN_Layer.py`, `RevIN.py`

Contents

1 Notation and Conventions **2**

2 `RevIN.py` — Reversible Instance Normalisation **2**
- 2.1 Input/output shapes . . . 3
- 2.2 Statistics computation (`_get_statistics`, lines 18–26) . . . 3
- 2.3 Normalisation (`_normalize`, lines 28–34) . . . 4
- 2.4 Denormalisation (`_denormalize`, lines 36–43) . . . 4

3 `ModernTCN_Layer.py` — Helper Modules **4**
- 3.1 `moving_avg` (lines 1–14) . . . 5
- 3.2 `series_decomp` (lines 17–23) . . . 6
- 3.3 `Flatten_Head` (lines 26–54) . . . 6

4 `ModernTCN.py` — Main Architecture **6**
- 4.1 Utility Functions (Lines 1–50) . . . 6
- 4.2 `ReparamLargeKernelConv` (Lines 52–124) . . . 8
- 4.3 `Block` (Lines 126–180) . . . 10
- 4.4 `Stage` (Lines 183–201) . . . 14
- 4.5 `ModernTCN` (Lines 204–344) . . . 14
- 4.6 `Model` (Lines 347–413) — Top-Level Wrapper . . . 19

5 End-to-End Forward Pass Summary **20**

Notation and Conventions

Throughout this document we use the following consistent notation, matching the variable names in the source code wherever possible.

Symbol	Code variable	Meaning
B	`B`	Batch size
M	`M` / `nvars` / `c_in`	Number of variables (channels)
L	`L` / `seq_len`	Input sequence length
T	`target_window`	Prediction length
D	`dmodel` / `dims[i]`	Feature (channel) dimension
r	`ffn_ratio`	FFN expansion ratio
rD	`dff`	Expanded hidden dimension
N	patch count	Number of temporal patches
P	`patch_size`	Patch size (stem kernel)
S	`patch_stride`	Patch stride
k_s	`large_size`	Large DWConv kernel size
k_s^{sm}	`small_size`	Small re-param kernel size
K	number of blocks	Number of ModernTCN blocks
μ, σ	`running_mean`, `running_var`	BN statistics
γ, β	`bn.weight`, `bn.bias`	BN affine parameters

All tensors are denoted in bold (e.g. $\mathbf{X}$). Scalars are italic (e.g. $x_{m,n}^{(d)}$ = entry of variable m, patch n, feature d). Tensor shapes are written in the order [batch $\times$ channels $\times$ time] for 3-D tensors and $[B \times M \times D \times N]$ for 4-D tensors.

`RevIN.py` — Reversible Instance Normalisation

`RevIN` (Kim et al., 2021) is a per-variable normalisation layer that removes the mean and variance of each variable *before* the backbone and restores them *after* prediction, mitigating distribution shift.

Listing 1: `RevIN.py` — full listing

```python
class RevIN(nn.Module):
    def __init__(self, num_features: int, eps=1e-5, affine=True,
                 subtract_last=False):
        super(RevIN, self).__init__()
        self.num_features = num_features   # = M (number of variables)
        self.eps = eps
        self.affine = affine
        self.subtract_last = subtract_last
        if self.affine:
            self._init_params()

    def _init_params(self):
        # Learnable scale and shift, one scalar per variable
        self.affine_weight = nn.Parameter(torch.ones(self.num_features))
        self.affine_bias   = nn.Parameter(torch.zeros(self.num_features))

    def _get_statistics(self, x):
        # x: [B, L, M]  -- time axis is dim 1
```

```
19          dim2reduce = tuple(range(1, x.ndim - 1))  # reduce over dim 1 (L)
20          if self.subtract_last:
21              self.last = x[:, -1, :].unsqueeze(1)  # last time-step [B,1,M]
22          else:
23              self.mean  = torch.mean(x, dim=dim2reduce, keepdim=True).
                   detach()
24              # mean: [B, 1, M]
25          self.stdev = torch.sqrt(
26              torch.var(x, dim=dim2reduce, keepdim=True, unbiased=False)
27              + self.eps).detach()
28          # stdev: [B, 1, M]
29
30      def _normalize(self, x):
31          x = x - (self.last if self.subtract_last else self.mean)
32          x = x / self.stdev
33          if self.affine:
34              x = x * self.affine_weight   # broadcast: [B,L,M] * [M]
35              x = x + self.affine_bias
36          return x
37
38      def _denormalize(self, x):
39          if self.affine:
40              x = x - self.affine_bias
41              x = x / (self.affine_weight + self.eps * self.eps)
42          x = x * self.stdev
43          x = x + (self.last if self.subtract_last else self.mean)
44          return x
45
46      def forward(self, x, mode: str):
47          if mode == 'norm':
48              self._get_statistics(x)
49              x = self._normalize(x)
50          elif mode == 'denorm':
51              x = self._denormalize(x)
52          return x
```

Input/output shapes

The input to `RevIN.forward` (in the context of `ModernTCN.forward`) is $\mathbf{X} \in \mathbb{R}^{B \times L \times M}$ after permuting from the model's internal $[B \times M \times L]$ layout.

Statistics computation (_get_statistics, lines 18–26)

For each sample b and variable m, the mean and standard deviation are computed along the time dimension $\ell \in \{1, \ldots, L\}$:

$$\hat{\mu}_{b,m} = \frac{1}{L} \sum_{\ell=1}^{L} x_{b,\ell,m}, \tag{1}$$

$$\hat{\sigma}_{b,m} = \sqrt{\frac{1}{L} \sum_{\ell=1}^{L} (x_{b,\ell,m} - \hat{\mu}_{b,m})^2 + \varepsilon}. \tag{2}$$

Both are kept as $[B \times 1 \times M]$ tensors (via `keepdim=True`) for broadcasting. The `.detach()` call stops gradients flowing through the statistics, treating them as constants during backpropagation.

Normalisation (`_normalize`, lines 28–34)

$$\hat{x}_{b,\ell,m} = \frac{x_{b,\ell,m} - \hat{\mu}_{b,m}}{\hat{\sigma}_{b,m}} \cdot \gamma_m + \beta_m, \tag{3}$$

where γ_m, β_m are learnable scalars (`affine_weight`, `affine_bias`), one per variable. They allow the network to rescale the normalised series per variable. The output $\hat{\mathbf{X}} \in \mathbb{R}^{B \times L \times M}$ has zero mean and unit variance per variable (before the affine transform).

Denormalisation (`_denormalize`, lines 36–43)

The inverse applies the same steps in reverse order, using the statistics saved during the `norm` call:

$$\tilde{x}_{b,t,m} = \left(\frac{\hat{x}_{b,t,m} - \beta_m}{\gamma_m + \varepsilon^2} \right) \cdot \hat{\sigma}_{b,m} + \hat{\mu}_{b,m}. \tag{4}$$

The ε^2 guard prevents division by near-zero γ_m.

`ModernTCN_Layer.py` — Helper Modules

Listing 2: `ModernTCN_Layer.py` — full listing

```python
class moving_avg(nn.Module):
    def __init__(self, kernel_size, stride):
        super(moving_avg, self).__init__()
        self.kernel_size = kernel_size
        self.avg = nn.AvgPool1d(kernel_size=kernel_size, stride=stride,
                                padding=0)

    def forward(self, x):           # x: [B, L, M]
        # Replicate-pad on both ends to preserve length
        front = x[:, 0:1, :].repeat(1, (self.kernel_size - 1) // 2, 1)
        end   = x[:, -1:, :].repeat(1, (self.kernel_size - 1) // 2, 1)
        x = torch.cat([front, x, end], dim=1)
        x = self.avg(x.permute(0, 2, 1))    # AvgPool expects [B,C,L]
        x = x.permute(0, 2, 1)
        return x

class series_decomp(nn.Module):
    def __init__(self, kernel_size):
        super(series_decomp, self).__init__()
        self.moving_avg = moving_avg(kernel_size, stride=1)

    def forward(self, x):
        moving_mean = self.moving_avg(x)     # trend component
        res = x - moving_mean                # residual (seasonal)
            component
        return res, moving_mean

class Flatten_Head(nn.Module):
```

```
28      def __init__(self, individual, n_vars, nf, target_window,
29                   head_dropout=0):
30          super(Flatten_Head, self).__init__()
31          self.individual = individual
32          self.n_vars = n_vars
33          if self.individual:
34              self.linears  = nn.ModuleList()
35              self.dropouts = nn.ModuleList()
36              self.flattens = nn.ModuleList()
37              for i in range(self.n_vars):
38                  self.flattens.append(nn.Flatten(start_dim=-2))
39                  self.linears.append(nn.Linear(nf, target_window))
40                  self.dropouts.append(nn.Dropout(head_dropout))
41          else:
42              self.flatten = nn.Flatten(start_dim=-2)
43              self.linear  = nn.Linear(nf, target_window)
44              self.dropout = nn.Dropout(head_dropout)
45
46      def forward(self, x):    # x: [B, M, D, N]
47          if self.individual:
48              x_out = []
49              for i in range(self.n_vars):
50                  z = self.flattens[i](x[:, i, :, :])   # [B, D*N]
51                  z = self.linears[i](z)                 # [B, T]
52                  z = self.dropouts[i](z)
53                  x_out.append(z)
54              x = torch.stack(x_out, dim=1)              # [B, M, T]
55          else:
56              x = self.flatten(x)     # [B, M, D*N]
57              x = self.linear(x)      # [B, M, T]
58              x = self.dropout(x)
59          return x
```

moving_avg (lines 1–14)

Padding (lines 9–11). To preserve the output length at L after average pooling with kernel size k, the series is replicated-padded on both ends by $p = (k-1)/2$ steps. For each variable m and batch b, the padded sequence is:

$$x^{\text{pad}}_{b,\ell,m} = \begin{cases} x_{b,1,m} & \ell \le p, \\ x_{b,\ell-p,m} & p < \ell \le L+p, \\ x_{b,L,m} & \ell > L+p. \end{cases} \tag{5}$$

The total padded length is $L + 2p = L + k - 1$, so after pooling with stride 1 the output length is $(L + k - 1) - k + 1 = L$.

Averaging (line 12). `AvgPool1d` computes, for each output position ℓ:

$$\bar{x}_{b,\ell,m} = \frac{1}{k} \sum_{\tau=0}^{k-1} x^{\text{pad}}_{b,\,\ell+\tau,\,m}. \tag{6}$$

The `permute` on line 12 converts $[B, L, M]$ to $[B, M, L]$ before pooling (which expects the time axis last), then permutes back on line 13.

`series_decomp` (lines 17–23)

The output of `moving_avg` is the *trend* component. The *residual* (seasonal) component is:

$$\mathbf{X}^{\text{res}} = \mathbf{X} - \bar{\mathbf{X}}, \qquad \mathbf{X}^{\text{trend}} = \bar{\mathbf{X}}. \tag{7}$$

Both retain shape $[B, L, M]$. This is used in `Model.forward` when `decomposition=True`, where two separate ModernTCN instances forecast the residual and trend independently.

`Flatten_Head` (lines 26–54)

Shared-weight mode (`individual=False`). After the backbone, $\mathbf{Z} \in \mathbb{R}^{B\times M\times D\times N}$. The flatten operation merges the last two axes:

$$\mathbf{Z}^{\text{flat}} = \text{Flatten}_{[-2,-1]}(\mathbf{Z}) \;\in\; \mathbb{R}^{B\times M\times (D\cdot N)}. \tag{8}$$

A single linear layer with weight $\mathbf{W}^{\text{head}} \in \mathbb{R}^{(DN)\times T}$ and bias $\mathbf{b}^{\text{head}} \in \mathbb{R}^{T}$ maps each of the M variables:

$$\hat{\mathbf{X}}_{b,m,\cdot} = \mathbf{Z}^{\text{flat}}_{b,m,\cdot}\,\mathbf{W}^{\text{head}} + \mathbf{b}^{\text{head}} \;\in\; \mathbb{R}^{T}. \tag{9}$$

The *same* weights are shared across all M variables.

Independent-weight mode (`individual=True`). Each variable m has its own linear map $\mathbf{W}^{\text{head}}_{m} \in \mathbb{R}^{(DN)\times T}$, applied via a loop. This doubles parameter count but allows each variable's forecast to be independent.

`ModernTCN.py` — Main Architecture

Utility Functions (Lines 1–50)

Listing 3: `ModernTCN.py` lines 1–50 — imports and utility functions

```
1  import torch
2  from torch import nn
3  import torch.nn.functional as F
4  import math
5  from layers.RevIN import RevIN
6  from models.ModernTCN_Layer import series_decomp, Flatten_Head
7
8  class LayerNorm(nn.Module):
9      def __init__(self, channels, eps=1e-6, data_format="channels_last"):
10         super(LayerNorm, self).__init__()
11         self.norm = nn.Layernorm(channels)   # note: typo in source ('L'
               cap)
12
13     def forward(self, x):
14         B, M, D, N = x.shape
15         x = x.permute(0, 1, 3, 2)   # [B, M, N, D]
16         x = x.reshape(B * M, N, D)  # [B*M, N, D]
17         x = self.norm(x)             # LayerNorm over last dim D
18         x = x.reshape(B, M, N, D)
19         x = x.permute(0, 1, 3, 2)   # [B, M, D, N]
20         return x
```

```python

def get_conv1d(in_channels, out_channels, kernel_size, stride, padding,
               dilation, groups, bias):
    return nn.Conv1d(in_channels=in_channels, out_channels=out_channels,
                     kernel_size=kernel_size, stride=stride,
                     padding=padding, dilation=dilation,
                     groups=groups, bias=bias)

def get_bn(channels):
    return nn.BatchNorm1d(channels)

def conv_bn(in_channels, out_channels, kernel_size, stride, padding,
            groups, dilation=1, bias=False):
    if padding is None:
        padding = kernel_size // 2   # 'same' padding for odd kernels
    result = nn.Sequential()
    result.add_module('conv',
        get_conv1d(in_channels, out_channels, kernel_size,
                   stride, padding, dilation, groups, bias))
    result.add_module('bn', get_bn(out_channels))
    return result

def fuse_bn(conv, bn):
    kernel       = conv.weight          # shape: [C_out, C_in/g, k]
    running_mean = bn.running_mean      # shape: [C_out]
    running_var  = bn.running_var       # shape: [C_out]
    gamma        = bn.weight            # shape: [C_out]
    beta         = bn.bias              # shape: [C_out]
    eps          = bn.eps
    std = (running_var + eps).sqrt()    # sigma_c for each channel c
    t   = (gamma / std).reshape(-1, 1, 1) # scale factor, broadcast over k
    return kernel * t, beta - running_mean * gamma / std
```

`LayerNorm` **(lines 8–20).** This applies PyTorch's `LayerNorm` over the feature dimension D of the 4-D tensor $[B \times M \times D \times N]$. The permute-reshape sequence exposes D as the last axis (required by `nn.LayerNorm`):

$$[B, M, D, N] \xrightarrow{\text{perm.}} [B, M, N, D] \xrightarrow{\text{resh.}} [BM, N, D]$$
$$\xrightarrow{\text{LN}_D} [BM, N, D] \xrightarrow{\text{resh.+perm.}} [B, M, D, N].$$

Note: this class is defined but not used in the released `ModernTCN.py`; `BatchNorm1d` is used instead.

`conv_bn` **(lines 31–38).** A helper that bundles a 1-D convolution and Batch Normalisation into a `Sequential`. The default padding `kernel_size // 2` ensures that an odd-kernel convolution preserves the temporal length N (as derived in the re-parameterisation document).

`fuse_bn` **(lines 40–50).** A **Conv1d–BN** sequence is mathematically equivalent to a single affinely-rescaled convolution. For channel c, define $\sigma_c = \sqrt{v_c + \varepsilon}$ where v_c is the running variance.

Then:

$$\tilde{w}_{c,\tau} = \frac{\gamma_c}{\sigma_c} w_{c,\tau}, \qquad \tau = 1, \ldots, k, \tag{10}$$

$$\tilde{b}_c = \beta_c - \frac{\gamma_c \mu_c}{\sigma_c}. \tag{11}$$

In code: `t = (gamma/std).reshape(-1,1,1)` gives the per-channel scale γ_c/σ_c broadcast over the kernel axis k. Then `kernel * t` gives $\tilde{\mathbf{W}}$ and `beta - running_mean * gamma / std` gives $\tilde{\mathbf{b}}$.

`ReparamLargeKernelConv` (Lines 52–124)

Listing 4: `ModernTCN.py` lines 52–124 — `ReparamLargeKernelConv`

```
52 class ReparamLargeKernelConv(nn.Module):
53
54     def __init__(self, in_channels, out_channels, kernel_size,
55                  stride, groups, small_kernel,
56                  small_kernel_merged=False, nvars=7):
57         super(ReparamLargeKernelConv, self).__init__()
58         self.kernel_size  = kernel_size     # k_s  (e.g. 51)
59         self.small_kernel = small_kernel    # k_sm (e.g.  5)
60         padding = kernel_size // 2          # p = (k_s-1)/2 for 'same'
               output
61         if small_kernel_merged:
62             # Inference mode: single fused conv with bias
63             self.lkb_reparam = nn.Conv1d(in_channels, out_channels,
64                 kernel_size=kernel_size, stride=stride, padding=padding,
65                 dilation=1, groups=groups, bias=True)
66         else:
67             # Training mode: two parallel Conv-BN branches
68             self.lkb_origin = conv_bn(in_channels, out_channels,
69                 kernel_size=kernel_size, stride=stride, padding=padding,
70                 dilation=1, groups=groups, bias=False)
71             if small_kernel is not None:
72                 assert small_kernel <= kernel_size
73                 self.small_conv = conv_bn(in_channels, out_channels,
74                     kernel_size=small_kernel, stride=stride,
75                     padding=small_kernel // 2,
76                     groups=groups, dilation=1, bias=False)
77
78     def forward(self, inputs):
79         if hasattr(self, 'lkb_reparam'):    # inference path
80             out = self.lkb_reparam(inputs)
81         else:                               # training path
82             out = self.lkb_origin(inputs)
83             if hasattr(self, 'small_conv'):
84                 out += self.small_conv(inputs)  # add parallel branch
85         return out
86
87     def PaddingTwoEdge1d(self, x, pad_length_left, pad_length_right,
88                          pad_values=0):
89         D_out, D_in, ks = x.shape         # kernel weight tensor
90         if pad_values == 0:
```

```
91                pad_left  = torch.zeros(D_out, D_in, pad_length_left)
92                pad_right = torch.zeros(D_out, D_in, pad_length_right)
93            else:
94                pad_left  = torch.ones(D_out, D_in, pad_length_left) *
                     pad_values
95                pad_right = torch.ones(D_out, D_in, pad_length_right)*
                     pad_values
96            x = torch.cat([pad_left,  x],      dims=-1)
97            x = torch.cat([x, pad_right],      dims=-1)
98            return x
99
100       def get_equivalent_kernel_bias(self):
101           # Step 1: fuse BN into each branch's conv kernel
102           eq_k, eq_b = fuse_bn(self.lkb_origin.conv, self.lkb_origin.bn)
103           if hasattr(self, 'small_conv'):
104               small_k, small_b = fuse_bn(self.small_conv.conv,
105                                          self.small_conv.bn)
106               eq_b += small_b   # add biases
107               delta = (self.kernel_size - self.small_kernel) // 2
108               eq_k += self.PaddingTwoEdge1d(small_k, delta, delta, 0)
109               # zero-pad small kernel to size k_s, then add element-wise
110           return eq_k, eq_b
111
112       def merge_kernel(self):
113           eq_k, eq_b = self.get_equivalent_kernel_bias()
114           self.lkb_reparam = nn.Conv1d(
115               in_channels  = self.lkb_origin.conv.in_channels,
116               out_channels = self.lkb_origin.conv.out_channels,
117               kernel_size  = self.lkb_origin.conv.kernel_size,
118               stride       = self.lkb_origin.conv.stride,
119               padding      = self.lkb_origin.conv.padding,
120               dilation     = self.lkb_origin.conv.dilation,
121               groups       = self.lkb_origin.conv.groups, bias=True)
122           self.lkb_reparam.weight.data = eq_k
123           self.lkb_reparam.bias.data   = eq_b
124           del self.lkb_origin              # remove training branches
125           if hasattr(self, 'small_conv'):
126               del self.small_conv
```

Constructor (lines 54–73). When `small_kernel_merged=False` (training), two parallel Conv1d-BN branches are created:

- `lkb_origin`: kernel size k_s, padding $p_L = k_s \,//\, 2$.
- `small_conv`: kernel size k_s^{sm}, padding $p_S = k_s^{\text{sm}} \,//\, 2$.

Both paddings satisfy the "same" condition $N_{\text{out}} = N$ (derived in the re-parameterisation document). When `small_kernel_merged=True` (inference), only the single fused `lkb_reparam` is present.

Forward pass (lines 76–84). During training:

$$\mathbf{Z}_{\text{dw}} = \text{BN}_L\big(\mathbf{W}_L \star \mathbf{Z}'\big) + \text{BN}_S\big(\mathbf{W}_S \star \mathbf{Z}'\big), \tag{12}$$

where $\star$ denotes the grouped depthwise convolution, and both $\mathrm{BN}_L, \mathrm{BN}_S$ are independent Batch Normalisation layers. During inference, this reduces to a single Conv1d with fused weights.

`PaddingTwoEdge1d` **(lines 87–98).** This pads the *kernel weight tensor* $\tilde{\mathbf{W}}_S \in \mathbb{R}^{C_{\text{out}} \times 1 \times k_s^{\text{sm}}}$ (the depthwise case has $C_{\text{in}}/g = 1$) with δ zeros on each side to produce a tensor of shape $\mathbb{R}^{C_{\text{out}} \times 1 \times k_s}$:

$$\tilde{\mathbf{W}}_S^{\text{pad}} = [\underbrace{\mathbf{0}}_{\delta}, \tilde{\mathbf{W}}_S, \underbrace{\mathbf{0}}_{\delta}], \quad \delta = \frac{k_s - k_s^{\text{sm}}}{2}. \tag{13}$$

For the default settings ($k_s = 51$, $k_s^{\text{sm}} = 5$): $\delta = 23$. This zero-padding is on the *kernel*, not on the input.

`get_equivalent_kernel_bias` **(lines 100–111).**

1. Fuse BN into each branch: $(\tilde{\mathbf{W}}_L, \tilde{\mathbf{b}}_L) = \texttt{fuse_bn}(\ldots)$ and $(\tilde{\mathbf{W}}_S, \tilde{\mathbf{b}}_S) = \texttt{fuse_bn}(\ldots)$.
2. Add biases: $\tilde{\mathbf{b}}_{\text{eq}} = \tilde{\mathbf{b}}_L + \tilde{\mathbf{b}}_S$.
3. Zero-pad small kernel: $\tilde{\mathbf{W}}_S^{\text{pad}}$.
4. Add kernels: $\tilde{\mathbf{W}}_{\text{eq}} = \tilde{\mathbf{W}}_L + \tilde{\mathbf{W}}_S^{\text{pad}}$.

The full fused weight matrix for all MD channels is:

$$\tilde{\mathbf{W}}_{\text{eq}} = \begin{bmatrix} \tilde{w}_{L,1}^{(1)} + \tilde{w}^{(1)S,1+\delta} & \cdots & \tilde{w}_{L,k_s}^{(1)} + \tilde{w}_{S,k_s^{\text{sm}}+\delta}^{(1)} \\ \vdots & \ddots & \vdots \\ \tilde{w}_{L,1}^{(MD)} + \tilde{w}^{(MD)S,1+\delta} & \cdots & \tilde{w}_{L,k_s}^{(MD)} + \tilde{w}_{S,k_s^{\text{sm}}+\delta}^{(MD)} \end{bmatrix} \in \mathbb{R}^{MD \times k_s}, \tag{14}$$

where the superscript on $\tilde{w}_{S,\tau}^{(c)}$ refers to the zero-padded position.

`merge_kernel` **(lines 113–124).** Creates the inference `lkb_reparam` Conv1d, loads the fused weights, and deletes the training branches to save memory.

`Block` (Lines 126–180)

Listing 5: `ModernTCN.py` lines 126–180 — `Block`

```
126 class Block(nn.Module):
127     def __init__(self, large_size, small_size, dmodel, dff, nvars,
128                  small_kernel_merged=False, drop=0.1):
129         super(Block, self).__init__()
130         # DWConv: depthwise, one filter per channel
131         self.dw = ReparamLargeKernelConv(
132             in_channels=nvars * dmodel, out_channels=nvars * dmodel,
133             kernel_size=large_size, stride=1,
134             groups=nvars * dmodel,      # fully depthwise (groups =
                    channels)
135             small_kernel=small_size,
136             small_kernel_merged=small_kernel_merged, nvars=nvars)
137         self.norm = nn.BatchNorm1d(dmodel)
138
```

```python
        # ConvFFN1: feature mixing per variable (groups = M)
        self.ffn1pw1 = nn.Conv1d(nvars * dmodel, nvars * dff,
                                 kernel_size=1, stride=1,
                                 padding=0, dilation=1, groups=nvars)
        self.ffn1act  = nn.GELU()
        self.ffn1pw2  = nn.Conv1d(nvars * dff, nvars * dmodel,
                                  kernel_size=1, stride=1,
                                  padding=0, dilation=1, groups=nvars)
        self.ffn1drop1 = nn.Dropout(drop)
        self.ffn1drop2 = nn.Dropout(drop)

        # ConvFFN2: variable mixing per feature (groups = D)
        self.ffn2pw1 = nn.Conv1d(nvars * dmodel, nvars * dff,
                                 kernel_size=1, stride=1,
                                 padding=0, dilation=1, groups=dmodel)
        self.ffn2act  = nn.GELU()
        self.ffn2pw2  = nn.Conv1d(nvars * dff, nvars * dmodel,
                                  kernel_size=1, stride=1,
                                  padding=0, dilation=1, groups=dmodel)
        self.ffn2drop1 = nn.Dropout(drop)
        self.ffn2drop2 = nn.Dropout(drop)

        self.ffn_ratio = dff // dmodel   # = r

    def forward(self, x):
        input = x                          # save for residual; [B, M, D, N]
        B, M, D, N = x.shape

        # --- DWConv ---
        x = x.reshape(B, M * D, N)         # merge M,D -> [B, MD, N]
        x = self.dw(x)                     # depthwise conv; out [B, MD, N]
        x = x.reshape(B, M, D, N)
        x = x.reshape(B * M, D, N)         # flatten batch+vars for BN
        x = self.norm(x)                   # BN over D channels
        x = x.reshape(B, M, D, N)
        x = x.reshape(B, M * D, N)         # back to [B, MD, N] for FFN1

        # --- ConvFFN1 (feature mixing per variable) ---
        x = self.ffn1drop1(self.ffn1pw1(x)) # [B, M*rD, N]
        x = self.ffn1act(x)                  # GeLU
        x = self.ffn1drop2(self.ffn1pw2(x)) # [B, M*D, N]
        x = x.reshape(B, M, D, N)

        # --- Permute for ConvFFN2 (variable mixing per feature) ---
        x = x.permute(0, 2, 1, 3)           # [B, D, M, N]
        x = x.reshape(B, D * M, N)           # [B, DM, N]

        # --- ConvFFN2 ---
        x = self.ffn2drop1(self.ffn2pw1(x)) # [B, D*rM, N]
        x = self.ffn2act(x)
        x = self.ffn2drop2(self.ffn2pw2(x)) # [B, D*M, N]
        x = x.reshape(B, D, M, N)
        x = x.permute(0, 2, 1, 3)            # [B, M, D, N]
```

```
192
193         # --- Residual connection ---
194         x = input + x
195         return x
```

Constructor: `dff` and `dmodel` (lines 127–153). As established in the re-parameterisation document: `dmodel` $= D$ and `dff` $= rD$. The ratio $r =$ `dff` / `dmodel` is stored as `self.ffn_ratio`.

DWConv setup (lines 130–132). The DWConv is a `ReparamLargeKernelConv` with `groups = nvars * dmodel = MD`. Since groups equals the number of channels, each of the MD channels has *exactly one* dedicated filter of length k_s, giving a fully depthwise convolution.

ConvFFN1 setup (lines 136–142).

- `ffn1pw1`: `groups=nvars=M`, so the MD input channels are split into M groups of D each. Each group maps $D \to rD$ independently (feature mixing per variable). Weight: $\mathbf{W}_{\mathrm{F1}}^{(m)} \in \mathbb{R}^{rD\times D}$ per group m.
- `ffn1pw2`: same grouping, maps $rD \to D$. Weight: $\mathbf{W}_{\mathrm{F2}}^{(m)} \in \mathbb{R}^{D\times rD}$ per group m.

ConvFFN2 setup (lines 145–151).

- `ffn2pw1`: `groups=dmodel=D`, so the DM channels (after permute) are split into D groups of M each. Each group maps $M \to rM$ (variable mixing per feature). Weight: $\mathbf{W}_{\mathrm{V1}}^{(d)} \in \mathbb{R}^{rM\times M}$ per group d.
- `ffn2pw2`: maps $rM \to M$. Weight: $\mathbf{W}_{\mathrm{V2}}^{(d)} \in \mathbb{R}^{M\times rM}$ per group d.

Forward: DWConv (lines 156–164). Line 156: Save the input $\mathbf{Z}_i \in \mathbb{R}^{B\times M\times D\times N}$ for the residual connection.

Line 158: `x.reshape(B, M*D, N)` merges the variable and feature dimensions, producing $\mathbf{Z}_i' \in \mathbb{R}^{B\times(MD)\times N}$. Entry-wise:

$$(\mathbf{Z}_i')_{b,\,(m-1)D+d,\,n} = (\mathbf{Z}_i)_{b,\,m,\,d,\,n}. \tag{15}$$

Line 159: `self.dw(x)` applies `ReparamLargeKernelConv`. For channel $c = (m-1)D + d$ at position n:

$$(\mathbf{Z}_i'')_{b,c,n} = \mathrm{BN}_c\left(\sum_{\tau=1}^{k_s} w_{c,\tau}^{\mathrm{DW}} \cdot (\mathbf{Z}_i')_{b,c,n+\tau-1-p_L}\right), \tag{16}$$

where $p_L = k_s \,//\, 2$ is the input zero-padding. Output: $\mathbf{Z}_i'' \in \mathbb{R}^{B\times(MD)\times N}$.

Lines 160–162: Reshape to $[B, M, D, N]$, then to $[BM, D, N]$ to apply `BatchNorm1d(dmodel)`, which normalises over the D-channel axis for each of the BM independent sequences. For sample b and variable m:

$$\mathrm{BN}\big(z_{b,m,n}^{(d)}\big) = \gamma_d\,\frac{z_{b,m,n}^{(d)} - \mu_d}{\sqrt{v_d + \varepsilon}} + \beta_d, \tag{17}$$

where μ_d, v_d are running statistics over the D channels.

Lines 163–164: Reshape back to $[B, M, D, N]$ then to $[B, MD, N]$ for the ConvFFN1 pass.

Forward: ConvFFN1 (lines 166–169). **Line 166**: `ffn1pw1` expands channels $MD \to MrD$:

$$\mathbf{u}_{b,\cdot,n}^{(m)} = \text{Dropout}\left(\mathbf{W}_{\text{F1}}^{(m)} \begin{bmatrix} z_{b,m,n}^{(1)} \\ \vdots \\ z_{b,m,n}^{(D)} \end{bmatrix} + \mathbf{b}_{\text{F1}}^{(m)}\right) \in \mathbb{R}^{rD}, \quad m = 1, \ldots, M. \tag{18}$$

Line 167: `GELU` activation applied element-wise.
Line 168: `ffn1pw2` projects back $MrD \to MD$:

$$\mathbf{f}_{b,\cdot,n}^{(m)} = \text{Dropout}\Big(\mathbf{W}_{\text{F2}}^{(m)}\,\text{GeLU}(\mathbf{u}_{b,\cdot,n}^{(m)}) + \mathbf{b}_{\text{F2}}^{(m)}\Big) \in \mathbb{R}^{D}. \tag{19}$$

Line 169: Reshape $[B, MD, N] \to [B, M, D, N]$.

Forward: Permute for ConvFFN2 (lines 171–172). **Line 171**: `x.permute(0, 2, 1, 3)` on $[B, M, D, N]$ gives $[B, D, M, N]$.

Line 172: Reshape to $[B, DM, N]$. The channel index is now $c' = (d-1)M + m$, giving a block of M consecutive channels per feature d:

$$(\mathbf{Z}_i^{f'})_{b,\,(d-1)M+m,\,n} = f_{b,m,n}^{(d)}, \tag{20}$$

which aligns each group of M channels with the same feature d.

Forward: ConvFFN2 (lines 173–177). **Line 173**: `ffn2pw1` (groups= D) expands $DM \to DrM$:

$$\boldsymbol{\alpha}_{b,\cdot,n}^{(d)} = \text{Dropout}\left(\mathbf{W}_{\text{V1}}^{(d)} \begin{bmatrix} f_{b,1,n}^{(d)} \\ \vdots \\ f_{b,M,n}^{(d)} \end{bmatrix} + \boldsymbol{\beta}_{\text{V1}}^{(d)}\right) \in \mathbb{R}^{rM}. \tag{21}$$

The input vector gathers feature d from *all* M variables: this is the cross-variable mixing step.

Line 174: `GELU` element-wise.
Line 175: `ffn2pw2` projects back $DrM \to DM$:

$$\mathbf{v}_{b,\cdot,n}^{(d)} = \text{Dropout}\Big(\mathbf{W}_{\text{V2}}^{(d)}\,\text{GeLU}(\boldsymbol{\alpha}_{b,\cdot,n}^{(d)}) + \boldsymbol{\beta}_{\text{V2}}^{(d)}\Big) \in \mathbb{R}^{M}. \tag{22}$$

Line 176: Reshape $[B, DM, N] \to [B, D, M, N]$.
Line 177: `x.permute(0,2,1,3)` restores $[B, M, D, N]$.

Forward: Residual (line 179).

$$\mathbf{Z}_{i+1} = \mathbf{Z}_i + \mathbf{Z}_i^{v'} \in \mathbb{R}^{B \times M \times D \times N}. \tag{23}$$

Entry-wise: $(\mathbf{Z}_{i+1})_{b,m,d,n} = (\mathbf{Z}_i)_{b,m,d,n} + v_{b,m,n}^{(d)}$.

Stage (Lines 183–201)

Listing 6: ModernTCN.py lines 183–201 — Stage

```
183 class Stage(nn.Module):
184     def __init__(self, ffn_ratio, num_blocks, large_size, small_size,
185                  dmodel, dw_model, nvars,
186                  small_kernel_merged=False, drop=0.1):
187         super(Stage, self).__init__()
188         d_ffn = dmodel * ffn_ratio      # dff = rD
189         blks  = []
190         for i in range(num_blocks):     # stack K identical blocks
191             blk = Block(large_size=large_size, small_size=small_size,
192                         dmodel=dmodel, dff=d_ffn, nvars=nvars,
193                         small_kernel_merged=small_kernel_merged,
194                         drop=drop)
195             blks.append(blk)
196         self.blocks = nn.ModuleList(blks)
197
198     def forward(self, x):               # x: [B, M, D, N]
199         for blk in self.blocks:         # apply K blocks sequentially
200             x = blk(x)
201         return x
```

Constructor (lines 184–194). A Stage stacks $K =$ num_blocks identical Block instances. The expanded hidden size is computed once: d_ffn $= D \cdot r$.

Forward (lines 196–200). The K blocks are applied in sequence. Denoting the block output function as $\mathcal{B}$, for $k = 1, \ldots, K$:

$$\mathbf{Z}^{(k)} = \mathcal{B}_k\left(\mathbf{Z}^{(k-1)}\right), \qquad \mathbf{Z}^{(0)} = \mathbf{Z}_{\text{in}}. \tag{24}$$

All blocks share the same hyperparameters but have *independent weights*.

ModernTCN (Lines 204–344)

Listing 7: ModernTCN.py lines 204–344 — ModernTCN class

```
204 class ModernTCN(nn.Module):
205     def __init__(self, patch_size, patch_stride, stem_ratio,
206                  downsample_ratio, ffn_ratio, num_blocks, large_size,
207                  small_size, dims, dw_dims, nvars,
208                  small_kernel_merged=False, backbone_dropout=0.1,
209                  head_dropout=0.1, use_multi_scale=True, revin=True,
210                  affine=True, subtract_last=False, freq=None,
211                  seq_len=512, c_in=7, individual=False,
212                  target_window=96):
213         super(ModernTCN, self).__init__()
214
215         # --- RevIN ---
216         self.revin = revin
217         if self.revin:
```

```
            self.revin_layer = RevIN(c_in, affine=affine,
                                     subtract_last=subtract_last)

        # --- Stem + Downsample layers ---
        self.downsample_layers = nn.ModuleList()
        stem = nn.Sequential(
            nn.Conv1d(1, dims[0], kernel_size=patch_size,
                      stride=patch_stride),   # patchify embedding
            nn.BatchNorm1d(dims[0])
        )
        self.downsample_layers.append(stem)
        for i in range(3):                    # 3 further downsample stages
            downsample_layer = nn.Sequential(
                nn.BatchNorm1d(dims[i]),
                nn.Conv1d(dims[i], dims[i+1],
                          kernel_size=downsample_ratio,
                          stride=downsample_ratio),
            )
            self.downsample_layers.append(downsample_layer)
        self.patch_size      = patch_size
        self.patch_stride    = patch_stride
        self.downsample_ratio = downsample_ratio

        # --- Time encoding (not used in base long-term forecasting) ---
        if freq == 'h':
            time_feature_num = 4
        elif freq == 't':
            time_feature_num = 5
        else:
            raise NotImplementedError
        self.te_patch = nn.Sequential(
            nn.Conv1d(time_feature_num, time_feature_num,
                      kernel_size=patch_size, stride=patch_stride,
                      groups=time_feature_num),
            nn.Conv1d(time_feature_num, dims[0],
                      kernel_size=1, stride=1, groups=1),
            nn.BatchNorm1d(dims[0]))

        # --- Backbone stages ---
        self.num_stage = len(num_blocks)
        self.stages    = nn.ModuleList()
        for stage_idx in range(self.num_stage):
            layer = Stage(ffn_ratio, num_blocks[stage_idx],
                          large_size[stage_idx], small_size[stage_idx],
                          dmodel=dims[stage_idx], dw_model=dw_dims[stage_idx],
                          nvars=nvars,
                          small_kernel_merged=small_kernel_merged,
                          drop=backbone_dropout)
            self.stages.append(layer)

        # --- Multi-scale fusion layers ---
        self.use_multi_scale  = use_multi_scale
```

```
270         self.up_sample_ratio  = downsample_ratio
271         self.lat_layer        = nn.ModuleList()
272         self.smooth_layer     = nn.ModuleList()
273         self.up_sample_conv   = nn.ModuleList()
274         for i in range(self.num_stage):
275             align_dim = dims[-1]
276             lat    = nn.Conv1d(dims[i], align_dim, kernel_size=1,stride=1)
277             smooth = nn.Conv1d(align_dim, align_dim,
278                                kernel_size=3, stride=1, padding=1)
279             up_conv = nn.Sequential(
280                 nn.ConvTranspose1d(align_dim, align_dim,
281                     kernel_size=self.up_sample_ratio,
282                     stride=self.up_sample_ratio),
283                 nn.BatchNorm1d(align_dim))
284             self.lat_layer.append(lat)
285             self.smooth_layer.append(smooth)
286             self.up_sample_conv.append(up_conv)
287
288         # --- Head ---
289         patch_num     = seq_len // patch_stride
290         self.n_vars   = c_in
291         self.individual = individual
292         d_model = dims[-1]
293         if use_multi_scale:
294             self.head_nf = d_model * patch_num
295         else:
296             if patch_num % pow(downsample_ratio, self.num_stage-1) == 0:
297                 self.head_nf = d_model * patch_num // pow(
298                     downsample_ratio, self.num_stage-1)
299             else:
300                 self.head_nf = d_model * (patch_num //
301                     pow(downsample_ratio, self.num_stage-1) + 1)
302         self.head = Flatten_Head(self.individual, self.n_vars,
303                                  self.head_nf, target_window,
304                                  head_dropout=head_dropout)
```

RevIN (lines 213–216). `RevIN(c_in)` creates an instance normaliser for M variables. It is applied in `forward` before and after the backbone.

Stem layer (lines 219–224). The stem is a `Conv1d` with:

- `in_channels=1`: each of the M univariate series is processed independently through the same 1-D channel.
- `out_channels=dims[0]=D`: maps 1 channel to D feature channels, implementing the patchify embedding.
- `kernel_size=patch_size=P`, `stride=patch_stride=S`.

The resulting operation per variable is exactly the patchify embedding derived in the embedding document:

$$\mathbf{z}_{m,n} = \mathbf{W}_{\text{stem}}\,\mathbf{p}_{m,n} + \mathbf{b}_{\text{stem}} \in \mathbb{R}^D, \tag{25}$$

where $\mathbf{p}_{m,n} \in \mathbb{R}^P$ is the n-th patch of variable m. Output: $\mathbf{X}_{\text{emb}} \in \mathbb{R}^{M\times D\times N}$.

Downsample layers (lines 226–231). For multi-scale operation, 3 further stages reduce the temporal dimension by a factor of `downsample_ratio` each:

$$N^{(s)} = \frac{N^{(s-1)}}{\rho}, \quad \rho = \texttt{downsample_ratio}, \quad s = 1, 2, 3. \tag{26}$$

Each stage first normalises with BN, then applies a `Conv1d` with `kernel_size` = `stride` = ρ (a non-overlapping strided convolution), increasing the feature dimension: $D^{(s)} = \texttt{dims}[s]$.

Backbone stages (lines 251–256). One `Stage` object per scale level is created. Each stage has its own D, k_s, k_s^{sm}, and number of blocks, allowing the architecture to adapt across scales.

Multi-scale fusion layers (lines 258–276). Three types of layers are created for each scale:

- `lat_layer`: a 1×1 pointwise Conv1d that projects each scale's $D^{(s)}$-channel output to the *final* scale's dimension $D^{(\text{final})} = \texttt{dims[-1]}$: $\mathbf{W}_s^{\text{lat}} \in \mathbb{R}^{D^{(\text{final})} \times D^{(s)}}$.
- `smooth_layer`: a 3×1 Conv1d with `padding=1` (preserves N) to smooth the fused features.
- `up_sample_conv`: a `ConvTranspose1d` (transposed/fractionally-strided convolution) followed by BN, used to upsample the coarser-scale features back to the finest scale's temporal resolution.

Head size (lines 279–294). The number of patches at the finest scale is $N_0 = L \,//\, S$. If multi-scale fusion is used, the head receives all N_0 patches of the aligned $D^{(\text{final})}$-channel representation, so:

$$\texttt{head_nf} = D^{(\text{final})} \times N_0. \tag{27}$$

Without multi-scale fusion, the backbone uses only the deepest (coarsest) stage, giving $N_{\text{final}} = N_0/\texttt{downsample_ratio}^{K-1}$ patches, so $\texttt{head_nf} = D^{(\text{final})} \times N_{\text{final}}$.

Listing 8: `ModernTCN.py` lines 297–344 — forward methods

```
297     def up_sample(self, x, upsample_ratio):
298         _, _, _, N = x.shape
299         return F.upsample(x, size=N, scale_factor=upsample_ratio,
300                           mode='bilinear')
301 
302     def forward_feature(self, x, te=None):
303         B, M, L = x.shape
304         x = x.unsqueeze(-2)             # [B, M, 1, L] -- add channel dim
305         for i in range(self.num_stage):
306             B, M, D, N = x.shape
307             x = x.reshape(B * M, D, N)          # merge B,M for Conv1d
308             if i == 0:
309                 # Stem padding: replicate last value (P - S) times
310                 if self.patch_size != self.patch_stride:
311                     pad_len = self.patch_size - self.patch_stride
312                     pad = x[:, :, -1:].repeat(1, 1, pad_len)
313                     x = torch.cat([x, pad], dim=-1)
314             else:
315                 # Align N to be divisible by downsample_ratio
316                 if N % self.downsample_ratio != 0:
```

```
317                     pad_len = self.downsample_ratio - (N % self.
                            downsample_ratio)
318                     x = torch.cat([x, x[:, :, -pad_len:]], dim=-1)
319             x = self.downsample_layers[i](x)     # stem or downsample conv
320             _, D_, N_ = x.shape
321             x = x.reshape(B, M, D_, N_)          # restore [B, M, D', N']
322             x = self.stages[i](x)                # ModernTCN blocks
323         return x
324
325     def forward(self, x, te=None):
326         # x: [B, M, L]
327         if self.revin:
328             x = x.permute(0, 2, 1)              # [B, L, M] for RevIN
329             x = self.revin_layer(x, 'norm')
330             x = x.permute(0, 2, 1)              # [B, M, L]
331         x = self.forward_feature(x, te)          # [B, M, D, N']
332         x = self.head(x)                         # [B, M, T]
333         if self.revin:
334             x = x.permute(0, 2, 1)              # [B, T, M]
335             x = self.revin_layer(x, 'denorm')
336             x = x.permute(0, 2, 1)              # [B, M, T]
337         return x
338
339     def structural_reparam(self):
340         for m in self.modules():
341             if hasattr(m, 'merge_kernel'):
342                 m.merge_kernel()                  # fuse all DWConv
                        branches
```

`forward_feature` (lines 301–323). **Line 305**: `x.unsqueeze(-2)` inserts a size-1 channel dimension, transforming $[B, M, L] \to [B, M, 1, L]$. This represents each variable as a single-channel 1-D signal before the stem convolution.

Lines 308–309: `x.reshape(B*M, D, N)` merges the batch and variable dimensions so that the variable-independent Conv1d can process all BM signals simultaneously with shared weights.

Lines 310–314 (stem padding): Before the stem convolution, if $P \neq S$ (overlapping patches), the last value is replicated $P - S$ times to ensure $N = L \,//\, S$ patches:

$$\mathbf{x}_{b\cdot m}^{\text{pad}} = \big[\, x_{b,m}^{(1)},\ \ldots,\ x_{b,m}^{(L)},\ \underbrace{x_{b,m}^{(L)},\ \ldots,\ x_{b,m}^{(L)}}_{P-S} \,\big]. \tag{28}$$

Lines 316–318 (downsample padding): At deeper stages, N may not be divisible by `downsample_ratio`. The last `pad_len` time steps are replicated to round N up to the nearest multiple.

Line 319: Apply the i-th downsample layer (stem for $i = 0$, strided Conv1d for $i > 0$).

Lines 320–322: Reshape back to $[B, M, D', N']$ and pass through the i-th backbone `Stage`.

`forward` (lines 325–338). The full forward pass for a single `ModernTCN` instance:

1. **RevIN norm** (lines 328–331): Permute $[B, M, L] \to [B, L, M]$, apply per-variable normalisation (Eq. 3), permute back.

2. **Feature extraction** (line 332): Call `forward_feature` to obtain $\mathbf{Z} \in \mathbb{R}^{B \times M \times D \times N'}$.
3. **Head** (line 333): Flatten and linearly project to predictions $\hat{\mathbf{X}} \in \mathbb{R}^{B \times M \times T}$.
4. **RevIN denorm** (lines 335–338): Permute, apply inverse normalisation (Eq. 4), permute back.

`structural_reparam` (lines 341–344). Iterates over all submodules and calls `merge_kernel()` on any module that has that method (i.e., all `ReparamLargeKernelConv` instances). This fuses all training branches into single inference Conv1d layers.

`Model` (Lines 347–413) — Top-Level Wrapper

Listing 9: `ModernTCN.py` lines 347–413 — `Model` wrapper

```
347 class Model(nn.Module):
348     def __init__(self, configs):
349         super(Model, self).__init__()
350         # Extract hyperparameters from configs object
351         self.stem_ratio       = configs.stem_ratio
352         self.downsample_ratio = configs.downsample_ratio
353         self.ffn_ratio        = configs.ffn_ratio       # r
354         self.num_blocks       = configs.num_blocks      # K (list)
355         self.large_size       = configs.large_size      # k_s (list)
356         self.small_size       = configs.small_size      # k_sm (list)
357         self.dims             = configs.dims            # D per stage (
            list)
358         self.dw_dims          = configs.dw_dims
359
360         self.nvars            = configs.enc_in          # M
361         self.small_kernel_merged = configs.small_kernel_merged
362         self.drop_backbone    = configs.dropout
363         self.drop_head        = configs.head_dropout
364         self.use_multi_scale  = configs.use_multi_scale
365         self.revin            = configs.revin
366         self.affine           = configs.affine
367         self.subtract_last    = configs.subtract_last
368
369         self.freq             = configs.freq
370         self.seq_len          = configs.seq_len         # L
371         self.c_in             = self.nvars,             # note trailing
            comma!
372         self.individual       = configs.individual
373         self.target_window    = configs.pred_len        # T
374
375         self.kernel_size      = configs.kernel_size     # for
            decomposition
376         self.patch_size       = configs.patch_size      # P
377         self.patch_stride     = configs.patch_stride    # S
378
379         # Optional series decomposition
380         self.decomposition    = configs.decomposition
381         if self.decomposition:
```

```
382             self.decomp_module = series_decomp(self.kernel_size)
383             self.model_res  = ModernTCN(...)  # forecast residual
384             self.model_trend = ModernTCN(...) # forecast trend
385         else:
386             self.model = ModernTCN(...)       # standard single model
387
388     def forward(self, x, te=None):
389         if self.decomposition:
390             # x: [B, L, M]
391             res_init, trend_init = self.decomp_module(x)
392             # permute to [B, M, L] for ModernTCN internal convention
393             res_init, trend_init = (res_init.permute(0,2,1),
394                                     trend_init.permute(0,2,1))
395             if te is not None:
396                 te = te.permute(0, 2, 1)
397             res   = self.model_res(res_init, te)    # [B, M, T]
398             trend = self.model_trend(trend_init, te)# [B, M, T]
399             x = res + trend                          # superposition
400             x = x.permute(0, 2, 1)                  # [B, T, M]
401         else:
402             x = x.permute(0, 2, 1)                  # [B, M, L]
403             if te is not None:
404                 te = te.permute(0, 2, 1)
405             x = self.model(x, te)                    # [B, M, T]
406             x = x.permute(0, 2, 1)                  # [B, T, M]
407         return x
```

Constructor (lines 348–394). `Model` is the entry point called by the training framework. It reads all hyperparameters from a `configs` object and instantiates either:

- A single `ModernTCN` (default, `decomposition=False`).
- Two parallel `ModernTCN` instances for trend and residual forecasting (`decomposition=True`).

Note (line 372): `self.c_in = self.nvars,` contains a trailing comma, making `self.c_in` a 1-tuple `(M,)` rather than the integer M. This is a bug in the source code; the `c_in` argument to `ModernTCN` will be a tuple.

Forward with decomposition (lines 397–406). When `decomposition=True`, the input $\mathbf{X} \in \mathbb{R}^{B\times L\times M}$ is split into trend and residual via `series_decomp` (Eq. 7). Both components are forecast independently and added:

$$\hat{\mathbf{X}} = \hat{\mathbf{X}}^{\text{res}} + \hat{\mathbf{X}}^{\text{trend}}. \tag{29}$$

This additive decomposition mirrors the approach in Autoformer and N-BEATS.

Forward without decomposition (lines 407–412). The standard path: permute to $[B, M, L]$, run `self.model`, permute back to $[B, T, M]$ for output.

End-to-End Forward Pass Summary

For a batch $\mathbf{X} \in \mathbb{R}^{B\times L\times M}$ (standard time-first layout):

1. **Input permute**: $[B, L, M] \to [B, M, L]$.
2. **RevIN norm**: per-variable zero-mean unit-variance normalisation with learnable affine parameters.
3. **Unsqueeze**: $[B, M, L] \to [B, M, 1, L]$.
4. **Stem + BN**: $[BM, 1, L_{\text{pad}}] \to [BM, D, N]$ (patchify embedding, $N = L \,//\, S$).
5. Reshape to $[B, M, D, N]$; enter first `Stage`.
6. **Stage i, Block k**:
   (a) Reshape $\to [B, MD, N]$; DWConv (temporal mixing); BN; reshape $\to [B, MD, N]$.
   (b) ConvFFN1 (groups= M): feature mixing per variable.
   (c) Permute + reshape $\to [B, DM, N]$; ConvFFN2 (groups= D): variable mixing per feature.
   (d) Permute + reshape $\to [B, M, D, N]$; residual add.
7. For multi-scale: downsample $N \to N/\rho$ at each stage; lateral projections align dimensions; transposed convolutions upsample back for fusion.
8. **Flatten head**: $[B, M, D, N'] \to [B, M, DN'] \to [B, M, T]$.
9. **RevIN denorm**: restore per-variable mean and variance.
10. **Output permute**: $[B, M, T] \to [B, T, M]$.

Module	Input shape	Output shape
RevIN (norm)	$[B, L, M]$	$[B, L, M]$
Unsqueeze	$[B, M, L]$	$[B, M, 1, L]$
Stem Conv1d + BN	$[BM, 1, L']$	$[BM, D, N]$
Stage (K blocks)	$[B, M, D, N]$	$[B, M, D, N]$
Downsample Conv1d	$[BM, D, N]$	$[BM, D', N/\rho]$
Flatten + Linear head	$[B, M, D, N']$	$[B, M, T]$
RevIN (denorm)	$[B, T, M]$	$[B, T, M]$